

College Allocation Database Management System

1. Project Overview

1.1 Purpose

The College Allocation Database Management System is a comprehensive web application designed to manage the college seat allocation process for students. The system handles student registration, preference management, seat allocation based on ranks, and provides prediction tools for students to estimate their chances of admission.

1.2 Key Features

- **Student Registration & Authentication:** Secure login system with session management
 - **Preference Management:** Students can add and manage their college and department preferences
 - **Seat Matrix Management:** View and manage available seats across different categories
 - **Automated Allocation Algorithm:** Rank-based allocation system with multiple rounds
 - **Cutoff Rank Tracking:** Historical data tracking for opening and closing ranks
 - **Prediction Tool:** AI-powered prediction of potential college allocations based on ranks
 - **Status Management:** Float, Freeze, and Withdraw status tracking for students
 - **Previous Allocation History:** View past allocation results and cutoff ranks
-

2. Technology Stack

Backend:

- **Express.js 5.1.0**
- **Node.js**
- **PostgreSQL**
- **Prisma 6.17.1**

Frontend

- **Next.js 15.5.3**

3. System Architecture

3.1 Architecture Overview

The system follows a **three-tier architecture**:



4. Database Design

4.1 Entity Relationship Model

The database consists of **11 main entities** with well-defined relationships:

Core Entities

1. **Category** - Student reservation categories (General, SC, ST, OBC, etc.)
2. **College** - Educational institutions
3. **Department** - Academic departments
4. **Program** - Specific programs (combination of College + Department)
5. **Student** - Student information and credentials
6. **Preferences** - Student preference ordering
7. **Seat_Matrix** - Available seats per program and category
8. **Round** - Allocation rounds
9. **Allocation_Status** - Allocation results
10. **CutOff_ranks** - Historical cutoff data

4.2 Database Indexes

The system implements strategic indexes for performance optimization:

```
-- Preferences table indexes
CREATE INDEX "Preferences_student_id_preference_number_idx"
ON "Preferences"("student_id", "preference_number");

CREATE INDEX "Preferences_student_id_idx"
ON "Preferences"("student_id");
```

Benefits:

- Faster preference lookups during allocation
- Optimized student preference queries
- Improved join performance

5. Database Triggers

The system implements **two critical database triggers** to maintain data consistency and automate business logic.

5.1 Auto-Register Trigger

Purpose

Automatically sets `isRegistered = true` when a student sets their password for the first time.

Key Features

- **BEFORE UPDATE:** Executes before the update is committed
- **Conditional Logic:** Only activates when password changes
- **Null Safety:** Checks for non-empty password
- **State Preservation:** Allows manual `isRegistered` updates when password isn't changing

Use Cases

- First-time registration via mobile number login
- Password reset scenarios

5.2 Withdraw Cleanup Trigger

Purpose

Automatically deletes all allocation records when a student's status changes to "withdrawn".

Key Features

- **AFTER UPDATE:** Executes after the update is committed
- **Conditional Execution:** Only runs when status changes to "withdrawn"
- **Cascading Deletion:** Removes all allocation records for the student
- **Data Integrity:** Prevents orphaned allocation records

Use Cases

- Student withdrawal from allocation process
- Administrative withdrawal actions
- Bulk status updates

6. Transactions and Data Integrity

6.1 Transaction Implementation

The system uses **Prisma transactions** to ensure data consistency during critical operations.

6.2 Allocation Process Transaction

The allocation process uses a transaction to ensure atomicity when inserting allocation records:

Key Features:

- **Batch Processing:** Processes 500 records per batch for performance
- **Atomicity:** All batches succeed or fail together
- **Skip Duplicates:** Prevents duplicate allocation errors
- **Logging:** Comprehensive logging for audit trail

6.3 Status Update Transaction

The float status update uses transactions for multiple student updates:

Key Features:

- **Multiple Operations:** Combines multiple update operations
- **Conditional Execution:** Only includes operations with data
- **Atomic Updates:** All status changes succeed or fail together

8. Core Algorithms

8.1 Allocation Algorithm

The allocation algorithm is the **core business logic** of the system.

Algorithm Overview

1. Initialize seat matrix map (program_id_category_id → available_seats)
2. Fetch all float students (registered, status = "float")
3. Decrement seats for frozen students (already allocated)
4. Sort students by rank (general_rank for general category, category_rank for reserved)
5. For each student: a. Fetch preferences ordered by preference_number b. For each preference:
 - Check general category seats (category_id = 1)
 - If available: Allocate using general_rank
 - Else: Check category-specific seats
 - If available: Allocate using category_rank
 - Break on first successful allocation
6. Batch insert all allocations in transaction
7. Calculate cutoff ranks (min/max per program-category)
8. Insert cutoff ranks

Optimization Strategies

1. **Batch Processing:** 500 records per transaction batch
2. **Indexed Queries:** Preferences table indexed on (student_id, preference_number)
3. **Selective Field Loading:** Only fetches required fields

8.2 Prediction Algorithm

The prediction algorithm estimates potential allocations based on historical cutoff ranks.

Algorithm Steps

1. Get student ranks and category
2. Filter programs by college/department preferences
3. Query cutoff ranks where:
4. $\text{closing_rank} \geq \text{student's general_rank}$ OR
5. $\text{closing_rank} \geq \text{student's category_rank}$
6. Filter by previous year's data
7. Return programs with cutoff information

12. Performance Optimizations

12.1 Database Optimizations

Indexes

- `Preferences(student_id, preference_number)` : Composite index for preference lookups
- `Preferences(student_id)` : Single column index for student queries

Query Optimization

- Selective field loading using `select`
- Batch operations for bulk inserts
- Connection pooling via Prisma

12.2 Application Optimizations

Batch Processing

- Allocation inserts: 500 records per batch
- Student imports: 100 records per batch
- Preference imports: 100 records per batch

In-Memory Data Structures

- Seat matrix stored in Map for $O(1)$ lookups

- Reduces database queries during allocation

12.4 Logging and Monitoring

Allocation Logging

- Comprehensive file-based logging (`allocation_logs.txt`)
- Timestamped entries
- Error tracking with stack traces

15. Conclusion

The College Allocation Database Management System is a robust, scalable solution for managing college seat allocations. With its comprehensive database design, automated triggers, transaction support, and efficient allocation algorithm, the system provides a reliable foundation for managing large-scale allocation processes.

The use of modern technologies (Next.js, Prisma, PostgreSQL) ensures maintainability and performance, while the detailed logging and error handling provide operational visibility and reliability.
